JACKFRUIT PHASE-1

SOFTWARE REQUIREMENTS SPECIFICATION

Anti-Performance Throttling Booster for Applications and Gaming Systems

TEAM DETAILS

Team Name	Project ID	Team Member 1	Team Member 2	Team Member 3	Team Member 4
Infinte Loop	50	NAME: Hamsa M N SRN: PES2UG24AM060	NAME: Harshitha Vangaveti SRN: PES2UG24AM064	NAME: K Ashritha Reddy SRN: PES2UG24AM071	NAME: Ankita Vellara SRN: PES2UG24AM21

Prepared by: Infinte Loop

Organization: PES University, Department of Computer Science & Engineering

Course: UE24CS341A - Software Engineering, V Semester

Date created: 06 September 2026

Revision History

Name	Date	Reason For Changes	Version
Infinte Loop	06-09-2026	Initial SRS	1.0

Table of Contents

Section	Topic
1	Introduction
1.1	Purpose
1.2	Intended Audience
1.3	Product Scope
1.4	References
2	Overall Description
2.1	Product Perspective
2.2	Product Functions
2.3	User Classes and Characteristics
2.4	Operating Environment
2.5	Design and Implementation Constraints
2.6	Assumptions and Dependencies
3	External Interface Requirements
3.1	User Interfaces
3.2	Software Interfaces
3.3	Communications Interfaces
4	Analysis Models
5	System Features
6	Other Nonfunctional Requirements
7	Other Requirements
Appendix A	Glossary
Appendix B	Field Layouts
Appendix C	Requirement Traceability Matrix

1. Introduction

This Software Requirements Specification defines the requirements for Project 50, "Anti-Performance Throttling Booster for Applications and Gaming Systems," Version 1.0. It specifies the intended functionality, interfaces, constraints, performance expectations, safety and security requirements, and quality attributes of the proposed software tool.

This document is intended for the student development team, faculty evaluators, testers, project reviewers, and prospective users. Developers should use Sections 2-5 to understand system behavior and requirements. Testers should use the functional and nonfunctional requirements to derive validation cases. Reviewers can use Sections 1-2 to understand scope and system context.

The product is a software tool that detects and helps mitigate performance throttling in applications and gaming systems by monitoring system resources, background processes, and network usage. It provides real-time monitoring and user-controlled performance profiles.

The system will enable users:

- Monitor CPU, memory, process, and relevant network usage in real time.
- Detect conditions associated with application or game performance throttling.
- Identify background processes and resource usage that may affect the target workload.
- View alerts, detected conditions, and optimization recommendations.
- Select or configure user-controlled performance profiles.
- Apply controlled optimization actions to eligible background processes/resources.
- View monitoring status and maintain logs of detected events and actions.

- JACKFRUIT Phase-1 Software Requirements Specification template supplied for UE24CS341A - Software Engineering.
- JACKFRUIT Project 50 scope: Anti-Performance Throttling Booster for Applications and Gaming Systems.
- Project GitHub repository and project artifacts maintained by the team.

2. Overall Description

The product is a standalone desktop-oriented monitoring and optimization tool. It interacts with the host operating system to collect performance information and, where permitted, perform controlled optimization actions. It complements rather than replaces the operating system, applications, or games.

- Real-time performance monitoring.
- Performance-throttling detection.
- Background-process analysis.
- Network-usage monitoring.
- User profile creation, selection, and modification.
- Resource/process optimization.
- Alerts and recommendations.
- Event and action logging.

User Class	Characteristics	Main Functions
End User / Gamer	Basic to intermediate computer knowledge.	Monitor performance, select profiles, view alerts, apply optimization.
Developer / Project Team	Technical knowledge of software development and operating systems.	Configure, implement, test, maintain, and validate the system.
Faculty Evaluator / Tester	Technical reviewer.	Review requirements, execute tests, and inspect results.

- Desktop/laptop computer capable of running the target application or game.

- Windows-based environment is the primary development and demonstration target.
- Python and supporting libraries may be used for monitoring and application logic.
- Git and GitHub are used for source control and collaboration.

- Agile lifecycle shall be followed as required by the course.
- GitHub shall be used for version control, collaboration, and traceable individual contributions.
- CI/CD, code review, and security validation shall be incorporated into the development process.
- The system shall not terminate or modify critical system processes without safeguards.
- Optimization shall operate only within permissions granted by the operating system/user.
- Monitoring overhead should be kept low so the tool does not itself significantly degrade performance.

- The operating system exposes sufficient performance and process information.
- The user has required permissions for permitted monitoring and optimization actions.
- The monitored application/game runs on the same host system.
- Effectiveness depends on hardware, OS configuration, workload, and application behavior.
- Third-party libraries and development tools remain compatible with the implementation.

3. External Interface Requirements

The application shall provide a dashboard-oriented interface showing real-time system status and monitored performance information. It shall provide controls for starting/stopping monitoring, selecting profiles, viewing alerts, and initiating permitted optimization actions. Status messages shall clearly indicate success, warning, or failure.

- Operating-system APIs/libraries for CPU, memory, process, and network metrics.
- Process-management interfaces for permitted optimization operations.
- Local storage or lightweight database/file storage for profiles and logs.
- Testing and static-analysis tools used by the development workflow.
- GitHub for source-code version control and collaboration.

Core monitoring and optimization are intended to operate locally. If a web/local dashboard is implemented, HTTP/HTTPS may be used for communication. Network monitoring shall collect usage statistics and shall not intercept private application content. Communication shall avoid unnecessary exposure of system information.

4. Analysis Models

The initial logical interaction model is: User -> Monitoring Engine -> Throttling Detection -> Cause Analysis -> Profile/Optimization Engine -> Dashboard/Alerts.

Primary use cases: Start Monitoring, View Metrics, Detect Throttling, Analyze Cause, Select Profile, Apply Optimization, View Alerts/Logs, and Stop Monitoring.

A formal UML use-case diagram and detailed architecture diagrams should be added during the design phase and referenced from this section.

5. System Features

The functional requirements below are organized by major system features. Each requirement has a unique identifier so it can later be traced to architecture, design, code, and test cases.

Description and Priority: Collect and display current performance information while monitoring is active. Priority: High.

Stimulus/Response: User starts monitoring -> system initializes collection -> system samples metrics -> dashboard updates.

Functional Requirements:

- REQ-01: The system shall allow the user to start and stop real-time monitoring.
- REQ-02: The system shall display current CPU and memory utilization.
- REQ-03: The system shall display relevant process-level resource usage.
- REQ-04: The system shall refresh monitoring information automatically at a defined interval.

Description and Priority: Detect conditions indicating possible performance degradation or throttling. Priority: High.

Stimulus/Response: Monitoring data is collected -> detection logic evaluates configured criteria -> abnormal condition is identified -> system generates a throttling event and notifies the user.

Functional Requirements:

- REQ-05: The system shall evaluate monitored metrics against configured detection criteria.
- REQ-06: The system shall identify and report suspected performance-throttling conditions.
- REQ-07: The system shall record the time and relevant metrics associated with a detected event.
- REQ-08: The system shall avoid reporting a condition as certain when the available evidence only indicates a suspected event.

Description and Priority: Identify resource-intensive background processes and provide controlled optimization actions. Priority: High.

Stimulus/Response: System detects resource contention -> identifies eligible background processes -> user reviews recommendation -> user confirms action -> system applies permitted optimization -> result is displayed/logged.

Functional Requirements:

- REQ-09: The system shall list relevant running processes and their resource usage.
- REQ-10: The system shall identify eligible background processes that may affect performance.
- REQ-11: The system shall require user confirmation before disruptive optimization actions.
- REQ-12: The system shall report the result of each optimization action.

Description and Priority: Monitor network usage relevant to application/game performance. Priority: Medium.

Stimulus/Response: Monitoring is active -> system collects network-usage statistics -> system displays usage and flags unusual/high usage where configured.

Functional Requirements:

- REQ-13: The system shall display relevant network usage statistics.
- REQ-14: The system shall associate network usage information with the monitoring period.
- REQ-15: The system shall alert the user when configured network-usage conditions are exceeded.

Description and Priority: Provide configurable modes for different workloads. Priority: High.

Stimulus/Response: User opens profiles -> selects or configures a profile -> system loads profile settings -> monitoring/optimization uses those settings.

Functional Requirements:

- REQ-16: The system shall provide predefined performance profiles.
- REQ-17: The system shall allow users to select an active profile.
- REQ-18: The system shall allow supported profile settings to be modified.
- REQ-19: The system shall apply the active profile to relevant optimization decisions.

Description and Priority: Communicate detected issues, recommendations, and actions. Priority: Medium.

Stimulus/Response: System detects an event -> generates alert/recommendation -> user views it -> system records the event/action.

Functional Requirements:

- REQ-20: The system shall display alerts for relevant detected conditions.
- REQ-21: The system shall provide optimization recommendations when sufficient information is available.
- REQ-22: The system shall log detected events and optimization actions with timestamps.
- REQ-23: The system shall allow the user to view recent logs.

6. Other Nonfunctional Requirements

- The dashboard should update frequently enough to support real-time monitoring.
- Monitoring should introduce minimal additional CPU and memory overhead.
- Optimization actions should provide a visible result after completion or failure.
- The system should remain responsive while monitoring is active.

- Critical operating-system processes shall not be terminated or modified by automated optimization.
- Potentially disruptive actions shall require user confirmation.
- The system shall fail safely when an optimization operation is not permitted.
- The application shall clearly communicate errors rather than silently applying unsupported actions.

- Only permitted local system information shall be collected.
- Logs shall avoid unnecessary sensitive information.
- Access to privileged optimization operations shall respect operating-system permissions.
- Source code and dependencies shall be checked using appropriate security/static-analysis practices.
- The system shall not intercept private application content merely for network monitoring.

- Usability: controls and alerts shall be understandable to a typical end user.
- Reliability: monitoring and logging should continue correctly during normal operation.
- Maintainability: code shall be modular and documented sufficiently for team maintenance.
- Testability: functional components shall be independently testable where practical.
- Portability: OS-specific operations should be isolated to facilitate future support for other platforms.

- The user controls whether monitoring is active.
- Optimization shall respect the selected performance profile.
- User confirmation is required before potentially disruptive actions.
- System-critical processes are protected from unsafe optimization.
- Implemented functionality shall be traceable to requirements and test cases.

7. Other Requirements

The system shall support project documentation and traceability required by the JACKFRUIT Software Engineering course. Requirements, source code, tests, and changes shall be maintained in the project repository. The implementation should support future extension of detection rules, profiles, monitoring metrics, and optimization strategies.

Appendix A: Glossary

Term	Definition
Performance Throttling	Reduction in application/system performance caused by resource, thermal, process, power, or related constraints.
CPU	Central Processing Unit.
RAM	Random Access Memory.
Process	A running program instance managed by the operating system.

Profile	A user-selected set of performance and optimization settings.
Monitoring	Continuous or periodic collection and display of system/application metrics.

Appendix B: Field Layouts

The following logical fields represent information that may be stored/displayed by the application. Exact implementation types may be refined during design.

Field	Data Type	Mandatory	Description
Timestamp	DateTime	Y	Time at which a metric/event was recorded.
CPU Usage	Decimal	Y	CPU utilization percentage.
Memory Usage	Decimal	Y	Memory utilization percentage.
Process Name	String	Y	Name of the monitored process.
Process CPU Usage	Decimal	N	CPU usage attributed to a process.
Process Memory Usage	Decimal	N	Memory usage attributed to a process.
Network Usage	Decimal	N	Relevant network usage metric.
Profile Name	String	Y	Name of the active performance profile.
Event Type	String	Y	Type of detected condition or action.
Action Result	String	N	Outcome of an optimization action.

Report requirements: The monitoring/report view should include timestamp, current resource metrics, detected events, active profile, recommended/selected action, and action result where applicable.

Appendix C: Requirement Traceability Matrix

Sl. No.	Requirement ID	Brief Description	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID	System Test ID
1	REQ-01	Start/stop monitoring	TBD	TBD	TBD	TBD	TBD
2	REQ-02	Display CPU/memory	TBD	TBD	TBD	TBD	TBD
3	REQ-05	Evaluate throttling criteria	TBD	TBD	TBD	TBD	TBD
4	REQ-09	List process resource usage	TBD	TBD	TBD	TBD	TBD
5	REQ-13	Display network usage	TBD	TBD	TBD	TBD	TBD
6	REQ-16	Provide performance profiles	TBD	TBD	TBD	TBD	TBD
7	REQ-20	Display alerts	TBD	TBD	TBD	TBD	TBD
8	REQ-22	Log events/actions	TBD	TBD	TBD	TBD	TBD